cated VRAM if you have a discrete GPU. That is not a promise that every model will run; it is a useful baseline showing that experimentation does not require a workstation.

The four limits to your AI

Before looking at CPUs and GPUs individually, understand what your machine is being asked to do.

First, the model weights need memory. A 12B model quantised to around four bits may have a weight file of roughly 6-8GB, although real sizes vary with the quantisation scheme and additional tensors. Google's own Gemma 4 estimates show the scale clearly: its 12B model needs about 26.7GB at BF16, 13.4GB at 8-bit and around 6.7GB at Q4_0. The 31B model rises from about 17.5GB at Q4_0 to almost 70GB at BF16.

Second, inference needs extra working memory. The model is not merely sitting in RAM like a movie file. Your runtime allocates buffers, and the key-value or KV cache grows as context length increases. Hugging Face's model memory estimator warns that estimating weight size alone is not enough for actual inference and suggests that additional inference overhead can be significant.

Third, something has to perform the maths. CPUs can do it. GPUs usually do it much faster because they are designed for highly parallel operations. NPUs can also accelerate certain AI workloads, but support depends heavily on the model format and software stack.

Fourth, all those weights have to move through memory quickly. That makes memory bandwidth surprisingly important. Two systems with enough capacity to hold the same model may generate at noticeably different speeds because one can feed the processor or GPU much faster.

Keep these four questions in mind: Will it fit? Will it compute quickly? Can memory feed it quickly? And what happens when I increase context?

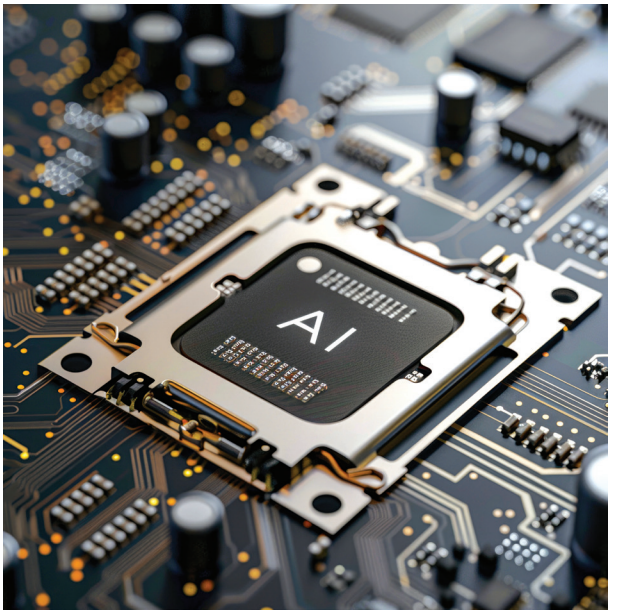
CPU: yes, you can run AI without a GPU

A dedicated GPU is useful, not mandatory. llama.cpp was designed specifically to make LLM inference practical across a wide range of hardware, including x86 CPUs with AVX-family instruction sets and ARM processors. It also supports numerous low-bit quantisation formats intended to reduce memory requirements and improve inference speed.

CPU-only inference is therefore an excellent way to begin, especially with small models. It is also useful when your GPU has little VRAM or your graphics hardware is unsupported.

The trade-off is speed. A CPU has relatively few powerful cores compared with the thousands of parallel execution units available to a modern GPU. Prompt processing can be slow, and larger models can reduce generation to an uncomfortable pace.

Do not judge CPUs only by core count. Modern instruction support, memory bandwidth, power limits and laptop cooling



matter. A desktop processor allowed to sustain high power can behave very differently from a thin-and-light CPU with the same marketing family name.

For CPU-only use, start small and quantised. Think of local AI as conversational software rather than a benchmark challenge. If a 4B model answers almost immediately and a 14B model makes you wait, the smaller model may be far more useful for everyday writing or summarisation.

GPU: acceleration meets the VRAM wall

For many PC users, the GPU is the biggest local-AI performance upgrade available. NVIDIA GPUs can use CUDA backends, AMD hardware can be accessed through technologies such as HIP/ROCm, and llama.cpp also offers Vulkan and other backends. Ollama supports NVIDIA and AMD acceleration and provides experimental Vulkan support for additional GPUs.

But GPU performance has a gatekeeper: VRAM.

A graphics processor may be extremely fast, but if the model

